

```
!pip install ultralytics opencv-python
```

```
Requirement already satisfied: ultralytics in /usr/local/lib/python3.12/dist-packages (8.4.54)
Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.2)
Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (3.10.0)
Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (11.3.0)
Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (6.0.3)
Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.32.4)
Requirement already satisfied: scipy>=1.4.1 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.16.3)
Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.10.0+cu128)
Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (0.25.0+cu128)
Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (5.9.5)
Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (1.35.2)
Requirement already satisfied: ultralytics-thop>=2.0.18 in /usr/local/lib/python3.12/dist-packages (from ultralytics) (2.0.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.0.7)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (4.22.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (1.4.0)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (24.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.3.0->ultralytics) (2.9.0)
Requirement already satisfied: polars-runtime-32==1.35.2 in /usr/local/lib/python3.12/dist-packages (from polars>=0.20.0->ultralytics) (1.35.2)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.23.0->ultralytics) (2025.1.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.29.0)
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (4.10.0)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (75.2.0)
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (1.13.3)
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.3.1)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2025.1.1)
Requirement already satisfied: cuda-bindings==12.9.4 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.9.4)
Requirement already satisfied: nvidia-cuda-nvrtc-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.93)
Requirement already satisfied: nvidia-cuda-runtime-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)
Requirement already satisfied: nvidia-cuda-cupti-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)
Requirement already satisfied: nvidia-cudnn-cu12==9.10.2.21 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (9.10.2.21)
Requirement already satisfied: nvidia-cublas-cu12==12.8.4.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.4.1)
Requirement already satisfied: nvidia-cufft-cu12==11.3.3.83 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.3.3.83)
Requirement already satisfied: nvidia-curand-cu12==10.3.9.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (10.3.9.90)
Requirement already satisfied: nvidia-cusolver-cu12==11.7.3.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (11.7.3.90)
Requirement already satisfied: nvidia-cusparselt-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.5.8.93)
Requirement already satisfied: nvidia-cusparse-cu12==12.5.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.5.8.93)
Requirement already satisfied: nvidia-cusparselt-cu12==0.7.1 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (0.7.1)
Requirement already satisfied: nvidia-nccl-cu12==2.27.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (2.27.5)
Requirement already satisfied: nvidia-nvshmem-cu12==3.4.5 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.4.5)
Requirement already satisfied: nvidia-nvtx-cu12==12.8.90 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.90)
Requirement already satisfied: nvidia-nvjitlink-cu12==12.8.93 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (12.8.93)
Requirement already satisfied: nvidia-cufile-cu12==1.13.1.3 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (1.13.1.3)
Requirement already satisfied: triton==3.6.0 in /usr/local/lib/python3.12/dist-packages (from torch>=1.8.0->ultralytics) (3.6.0)
Requirement already satisfied: cuda-pathfinder~=1.1 in /usr/local/lib/python3.12/dist-packages (from cuda-bindings==12.9.4->torch>=1.8.0->ultralytics) (1.1.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.3.0->ultralytics) (1.17.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.3->torch>=1.8.0->ultralytics) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->torch>=1.8.0->ultralytics) (2.1.5)
```

```
from ultralytics import YOLO
import cv2
import numpy as np
```

```
model = YOLO("yolov8n.pt")
```

```
import cv2
cap = cv2.VideoCapture("car3.mp4")
ret, frame = cap.read()
print(frame.shape)
```

```
(2160, 3840, 3)
```

```
from google.colab.patches import cv2_imshow
cv2_imshow(frame)
```



```
results = model(frame)
results[0].show()
```

0: 384x640 2 persons, 3 cars, 1 airplane, 2 trucks, 124.3ms

Speed: 9.4ms preprocess, 124.3ms inference, 46.4ms postprocess per image at shape (1, 3, 384, 640)



```
target_box = None
for box in results[0].boxes:
    cls = int(box.cls[0])
    if cls == 2:
        x1, y1, x2, y2 = box.xyxy[0]
        x1 = int(x1)
        y1 = int(y1)
        x2 = int(x2)
        y2 = int(y2)
        area = (x2 - x1) * (y2 - y1)
        if area > 2000 and area < 20000 and y1 < 1300:
            target_box = box
            cv2.rectangle(
                frame,
                (x1, y1),
                (x2, y2),
                (0, 255, 0),
                3
            )
            break
cv2.imshow('frame')
```




```
x1, y1, x2, y2 = target_box.xyxy[0]

prev_x = int(x1)
prev_y = int(y1)

tracker_confidence = float(target_box.conf[0])
motion_confidence = 0.75
trusted_source = "Tracker"
print("Bounding Box:")
print("x =", int(x1))
print("y =", int(y1))
print("width =", int(x2 - x1))
print("height =", int(y2 - y1))
print("Tracker Confidence =", tracker_confidence)
print("Motion Model Confidence =", motion_confidence)
print("Trusted Source =", trusted_source)
```

```
Bounding Box:
x = 1610
y = 1065
width = 126
height = 108
Tracker Confidence = 0.47405168414115906
Motion Model Confidence = 0.75
Trusted Source = Tracker
```

```
current_x = int(x1)
current_y = int(y1)

vx = current_x - prev_x
vy = current_y - prev_y

predicted_x = current_x + vx
predicted_y = current_y + vy

print("Velocity X =", vx)
print("Velocity Y =", vy)
```

```
print("Predicted Next X =", predicted_x)
print("Predicted Next Y =", predicted_y)
```

```
Velocity X = 0
Velocity Y = 0
Predicted Next X = 1610
Predicted Next Y = 1065
```

```
cap = cv2.VideoCapture("car3.mp4")

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

fourcc = cv2.VideoWriter_fourcc(*'mp4v')

out = cv2.VideoWriter(
    "output.mp4",
    fourcc,
    fps,
    (width, height)
)
```

```
import cv2
from ultralytics import YOLO
import math

model = YOLO("yolov8n.pt")

cap = cv2.VideoCapture("car3.mp4")

width = int(cap.get(cv2.CAP_PROP_FRAME_WIDTH))
height = int(cap.get(cv2.CAP_PROP_FRAME_HEIGHT))
fps = int(cap.get(cv2.CAP_PROP_FPS))

fourcc = cv2.VideoWriter_fourcc(*'mp4v')

out = cv2.VideoWriter(
    "output.mp4",
    fourcc,
    fps,
    (width, height)
)

ret, frame = cap.read()

if not ret:
    print("Video not found")
    exit()

results = model(frame)

target_box = None

for box in results[0].boxes:

    cls = int(box.cls[0])

    if cls == 2:

        x1, y1, x2, y2 = box.xyxy[0]

        width_box = x2 - x1
        height_box = y2 - y1

        area = width_box * height_box

        # select small distant car
        if 5000 < area < 30000:

            target_box = box
            break

x1, y1, x2, y2 = target_box.xyxy[0]

prev_x = int(x1)
prev_y = int(y1)

width_box = int(x2 - x1)
height_box = int(y2 - y1)

tracker_confidence = float(target_box.conf[0])
```

```

motion_confidence = 0.75

trusted_source = "Tracker"

print("Bounding Box:")

print("x =", int(x1))
print("y =", int(y1))

print("width =", width_box)
print("height =", height_box)

print("Tracker Confidence =", tracker_confidence)

print("Motion Model Confidence =", motion_confidence)

print("Trusted Source =", trusted_source)

while True:

    ret, frame = cap.read()

    if not ret:
        break

    results = model(frame)

    best_box = None

    minimum_distance = 999999

    for box in results[0].boxes:

        cls = int(box.cls[0])

        # ONLY CAR
        if cls != 2:
            continue

        x1, y1, x2, y2 = box.xyxy[0]

        x1 = int(x1)
        y1 = int(y1)

        x2 = int(x2)
        y2 = int(y2)

        current_width = x2 - x1
        current_height = y2 - y1

        # STRICT SIZE MATCH
        width_difference = abs(current_width - width_box)
        height_difference = abs(current_height - height_box)

        # if size changes too much -> reject
        if width_difference > 35:
            continue

        if height_difference > 35:
            continue

        # DISTANCE FROM PREVIOUS POSITION
        distance = math.sqrt(
            (x1 - prev_x) ** 2 +
            (y1 - prev_y) ** 2
        )

        # STRICT DISTANCE LIMIT
        if distance > 120:
            continue

        # CHOOSE CLOSEST BOX ONLY
        if distance < minimum_distance:

            minimum_distance = distance
            best_box = box

    # TRACK ONLY IF SAME VEHICLE FOUND
    if best_box is not None:
        x1, y1, x2, y2 = best_box.xyxy[0]

```

```

x1 = int(x1)
y1 = int(y1)

x2 = int(x2)
y2 = int(y2)

width_box = x2 - x1
height_box = y2 - y1

tracker_confidence = float(best_box.conf[0])

current_x = x1
current_y = y1

vx = current_x - prev_x
vy = current_y - prev_y

predicted_x = current_x + vx
predicted_y = current_y + vy

if tracker_confidence > 0.5:
    trusted_source = "Tracker"
else:
    trusted_source = "Motion Model"

cv2.rectangle(
    frame,
    (x1, y1),
    (x2, y2),
    (0, 255, 0),
    2
)

cv2.putText(
    frame,
    f"Tracker:{tracker_confidence:.2f}",
    (x1, y1 - 80),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0, 255, 0),
    2
)

cv2.putText(
    frame,
    f"Motion:{motion_confidence:.2f}",
    (x1, y1 - 50),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (0, 0, 255),
    2
)

cv2.putText(
    frame,
    f"Source:{trusted_source}",
    (x1, y1 - 20),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 255, 0),
    2
)

cv2.putText(
    frame,
    f"PredX:{predicted_x} PredY:{predicted_y}",
    (x1, y2 + 30),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.6,
    (255, 0, 0),
    2
)

# UPDATE ONLY SAME CAR
prev_x = x1
prev_y = y1

out.write(frame)

cap.release()
out.release()

```

```
print("Tracking Completed")
Speed: 4.5ms preprocess, 10.0ms inference, 1.0ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 3 cars, 1 truck, 18.1ms
Speed: 6.7ms preprocess, 18.1ms inference, 2.0ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 3 cars, 1 truck, 26.3ms
Speed: 6.6ms preprocess, 26.3ms inference, 3.1ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 truck, 25.1ms
Speed: 6.2ms preprocess, 25.1ms inference, 3.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 truck, 9.6ms
Speed: 4.9ms preprocess, 9.6ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 airplane, 1 truck, 8.7ms
Speed: 4.7ms preprocess, 8.7ms inference, 1.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 truck, 9.8ms
Speed: 6.1ms preprocess, 9.8ms inference, 1.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 truck, 9.6ms
Speed: 5.1ms preprocess, 9.6ms inference, 1.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 truck, 10.9ms
Speed: 5.0ms preprocess, 10.9ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 truck, 9.2ms
Speed: 6.3ms preprocess, 9.2ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 motorcycle, 1 truck, 10.8ms
Speed: 4.9ms preprocess, 10.8ms inference, 1.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 truck, 9.7ms
Speed: 5.6ms preprocess, 9.7ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 3 cars, 1 airplane, 1 truck, 9.8ms
Speed: 5.7ms preprocess, 9.8ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 3 cars, 1 truck, 8.9ms
Speed: 5.3ms preprocess, 8.9ms inference, 1.7ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 1 person, 3 cars, 1 airplane, 1 truck, 9.9ms
Speed: 6.3ms preprocess, 9.9ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 cars, 1 airplane, 1 truck, 9.5ms
Speed: 6.0ms preprocess, 9.5ms inference, 2.0ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 cars, 1 airplane, 1 truck, 10.7ms
Speed: 3.3ms preprocess, 10.7ms inference, 1.9ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 3 cars, 1 airplane, 1 truck, 9.2ms
Speed: 4.1ms preprocess, 9.2ms inference, 1.8ms postprocess per image at shape (1, 3, 384, 640)

0: 384x640 2 persons, 2 cars, 1 airplane, 1 truck, 10.0ms
Speed: 3.7ms preprocess, 10.0ms inference, 2.5ms postprocess per image at shape (1, 3, 384, 640)
```